

DBMS & Django Database -- Complete Notes

Part 1: Important DBMS Terms

1. Data Inconsistency

Definition

When the same logical data has ****different values in different places**** in a database, it is called ****Data Inconsistency****.

Example

One table:

```
```text
```

Faruk Age = 25

```
```
```

Another table:

```
```text
```

Faruk Age = 26

```
```
```

The same person's age is different.

Therefore, this is ****Data Inconsistency****.

Common Cause

Data inconsistency can occur because of:

- * Excessive data redundancy

- * Improper updates
- * Poor database design

2. Data Redundancy

Definition

When the same information is stored **more times than necessary** in a database, it is called **Data Redundancy**.

Example

Suppose a customer's address is stored repeatedly with 10 different orders:

```text

Order 1 Faruk Bogura

Order 2 Faruk Bogura

Order 3 Faruk Bogura

...

Order 10 Faruk Bogura

```

Problems

- * Wastes storage
- * Makes updates harder
- * Can cause inconsistency
- * Increases maintenance cost

3. Duplication

Definition

When the same data appears in multiple records or locations, it is commonly referred to as **Duplication**.

Example

```text

Faruk 01750000000

Ahmed 01750000000

```

The same phone number appears multiple times.

Important

Duplication and **redundancy** are closely related, but in database design, **redundancy** is the more important technical concept.

4. Anomaly

Definition

An **Anomaly** is an unwanted problem that occurs during database operations such as:

* INSERT

* UPDATE

* DELETE

Anomalies are commonly caused by poor database design and unnecessary redundancy.

Types

1. Insertion Anomaly
2. Update Anomaly
3. Deletion Anomaly

5. Insertion Anomaly

Definition

An **Insertion Anomaly** occurs when we cannot insert required data without providing unrelated data, or when inserting new data becomes unnecessarily difficult.

Example

Suppose we have:

```text

Student\_ID | Student\_Name | Course\_ID | Course\_Name

```

Now suppose we want to add a new course, but no student has enrolled yet.

If the table design requires student information before a course can be stored, we have an **Insertion Anomaly**.

6. Update Anomaly

Definition

An **Update Anomaly** occurs when the same information is stored in multiple places and an update is made in some places but not others.

Example

A teacher's department is stored in five records:

```text

Record 1 CSE

Record 2 CSE

Record 3 CSE

Record 4 CSE

Record 5 CSE

```

Suppose the department changes to EEE, but only four records are updated:

```text

Record 1 EEE

Record 2 EEE

Record 3 EEE

Record 4 EEE

Record 5 CSE

```

The database now contains inconsistent information.

This is an **Update Anomaly**.

7. Deletion Anomaly

Definition

A **Deletion Anomaly** occurs when deleting one piece of data unintentionally causes other

required information to be lost.

Example

Suppose a course's information exists only in the record of its only enrolled student.

If we delete that student record, we may also lose the course information.

This is a **Deletion Anomaly**.

8. Consistency

Definition

Consistency means that the database remains in a logically valid state according to its defined rules, constraints, and relationships.

Example

If a customer's address is updated, the database should continue to satisfy all relevant constraints and relationships.

Important

Consistency does not simply mean:

> "The same value must appear everywhere."

It means the database remains logically valid.

9. Data Integrity

Definition

****Data Integrity**** means ensuring that database data remains:

- * Valid
- * Accurate
- * Consistent
- * Reliable
- * Properly protected from invalid modifications

Example

Preventing a negative age:

```
``text
age >= 0
``
```

This helps maintain data integrity.

Common Types

- * Entity Integrity
- * Referential Integrity
- * Domain Integrity
- * Business/User-defined constraints

10. Accuracy

Definition

Data is ****accurate**** when it correctly represents the real or authoritative information.

Example

Actual customer age:

```
```text
```

25

```
```
```

Database:

```
```text
```

age = 25

```
```
```

Therefore, the data is accurate.

```
---
```

11. Completeness

Definition

****Completeness**** means that all information required for a particular business or application purpose is available.

Example

Suppose a customer requires:

```
```text
```

Name

Phone

Address

```
```
```

If all required information is available, the data is complete for that requirement.

12. Ambiguity

Definition

When data or a statement has more than one possible meaning, it is called ****Ambiguity****.

Example

```text

Java

```

It can mean:

* Java Programming Language

* Java Island

Without context, the term is ambiguous.

13. Dependency

Definition

When the value of one attribute depends on another attribute, it is called a ****Dependency****.

Example

```text

Student\_ID Student\_Name

```

If Student_ID uniquely determines Student_Name, this represents a **Functional Dependency**.

14. Normalization

Definition

Normalization is the process of organizing a database into logically structured tables to reduce redundancy and data anomalies.

Main Goals

- * Reduce redundancy
- * Reduce update anomalies
- * Reduce insertion anomalies
- * Reduce deletion anomalies
- * Improve data integrity

Common Normal Forms

``text

1NF

2NF

3NF

BCNF

4NF

5NF

...

15. Denormalization

Definition

****Denormalization**** is the intentional introduction of some redundancy or combining of data structures to improve performance for particular workloads.

Purposes

- * Improve read performance
- * Reduce expensive joins in some cases
- * Improve reporting/query performance

Trade-offs

Denormalization can increase:

- * Data redundancy
- * Storage requirements
- * Update complexity

Quick DBMS Revision

``text

Redundancy

Same information stored unnecessarily multiple times.

Inconsistency

Same logical data has different values.

Duplication

Same data appears in multiple places.

Anomaly

Unwanted problem during INSERT, UPDATE, or DELETE.

Insertion Anomaly

Problem while inserting data.

Update Anomaly

Problem while updating duplicated data.

Deletion Anomaly

Required data is lost while deleting other data.

Consistency

Database remains in a valid logical state.

Integrity

Data remains valid, accurate, reliable, and constraint-compliant.

Normalization

Organizing data to reduce redundancy and anomalies.

Denormalization

Intentionally introducing some redundancy for specific performance/use cases.

...

Part 2: Django Bulk Operations

What is a Bulk Operation?

A **bulk operation** means performing an operation on many database records together.

Common bulk operations include:

- * Bulk Insert
- * Bulk Update
- * Bulk Delete

1. Normal Update

```
```python
for participant in Participant.objects.all():
 participant.is_winner = False
 participant.save()
```
```

Here, each object is updated separately.

For a large number of records, this can result in many database operations.

2. Bulk Update

```
```python
Participant.objects.update(is_winner=False)
```
```

Conceptually, the database performs something like:

```
```sql
UPDATE participant
SET is_winner = FALSE;
```
```

Advantages

- * Updates many records at once
- * Avoids a Python loop
- * Uses a set-based database operation
- * Usually more efficient for mass updates

3. `bulk\_create()`

Used to insert many objects:

```
```python
Participant.objects.bulk_create([
 Participant(
 name='John',
 phone='123',
 email='john@example.com'
),
 Participant(
 name='Jane',
 phone='456',
 email='jane@example.com'
),
])
```
```

4. `bulk\_update()`

Used to update multiple existing model objects:

```
```python
participants = [
 Participant(id=1, name='John Updated'),
 Participant(id=2, name='Jane Updated'),
]

Participant.objects.bulk_update(
 participants,
 ['name']
)
```
```

The fields to update must be specified.

5. Bulk Delete

Django does not have a standard:

```
```python
Participant.objects.bulk_delete(...)
```
```

Instead, use QuerySet `delete()`:

```
```python
Participant.objects.filter(
 id__in=[1, 2]

```

```
).delete()
```

```
...
```

```

```

## # Bulk Operation Summary

```
| Operation | Django API |
```

```
| ----- | ----- |
```

```
| Bulk Insert | `bulk_create()` |
```

```
| Bulk Update | `bulk_update()` |
```

```
| QuerySet Update | `update()` |
```

```
| Bulk Delete | `QuerySet.delete()` |
```

```

```

## # Part 3: Django Model Relationships

Django model relationships define how database tables are related to each other.

There are three main relationship types:

```
| Relationship | Django Field | Example |
```

```
| ----- | ----- | ----- |
```

```
| One-to-One | `OneToOneField` | User Profile |
```

```
| One-to-Many / Many-to-One | `ForeignKey` | Category Products |
```

```
| Many-to-Many | `ManyToManyField` | Student Course |
```

```

```

### # 1. One-to-One Relationship

Use `OneToOneField` when one object is associated with one object in another model.

```

```python
from django.db import models
from django.contrib.auth.models import User

class Profile(models.Model):
    user = models.OneToOneField(
        User,
        on_delete=models.CASCADE
    )
    phone = models.CharField(max_length=15)
    address = models.TextField()
...

```

Relationship:

```

```text
User 1 1 Profile
...

```

---

## # 2. One-to-Many / Many-to-One

Example:

```

```python
class Category(models.Model):
    name = models.CharField(max_length=100)

class Product(models.Model):

```

```

category = models.ForeignKey(
    Category,
    on_delete=models.CASCADE
)
name = models.CharField(max_length=100)
price = models.DecimalField(
    max_digits=10,
    decimal_places=2
)
...

```

Relationship:

```

```text
Category 1 * Product
...

```

Meaning:

```

```text
One Category  Many Products
Many Products  One Category
...

```

3. Many-to-Many

```

```python
class Student(models.Model):
 name = models.CharField(max_length=100)

```

```
class Course(models.Model):
 name = models.CharField(max_length=100)
 students = models.ManyToManyField(Student)
 ...
```

Relationship:

```
```text
Student * * Course
...

```

A student can take many courses, and a course can have many students.

```
# `on_delete`
```

```
## `CASCADE`
```

When the referenced object is deleted, related objects may also be deleted.

```
```python
on_delete=models.CASCADE
...

```

---

```
`SET_NULL`
```

Sets the foreign key to `NULL` when the referenced object is deleted.

The field must allow NULL:

```
```python
```

```
author = models.ForeignKey(
    Author,
    null=True,
    on_delete=models.SET_NULL
)
'''
```

`PROTECT`

Prevents deletion of the referenced object when related objects exist.

```
'''python
on_delete=models.PROTECT
'''
```

`DO\_NOTHING`

Django does not perform any deletion handling.

```
'''python
on_delete=models.DO_NOTHING
'''
```

Database constraints may still cause an error.

How to Know When a Relationship Is Needed

When designing models, ask:

> "Is this entity logically related to another entity?"

Suppose we have:

```
```text
```

Shop

Product

Customer

Order

Profile

```
```
```

```
### Shop Product
```

One shop can have many products:

```
```python
```

```
shop = models.ForeignKey(
 Shop,
 on_delete=models.CASCADE
)
```

```
```
```

```
### Customer Order
```

One customer can have many orders:

```
```python
```

```
customer = models.ForeignKey(
 Customer,
 on_delete=models.CASCADE
```

```
)
...
```

#### ### Order Product

An order can contain many products, and a product can appear in many orders.

A simple design could use:

```
```python  
products = models.ManyToManyField(Product)  
...
```

However, in a real e-commerce system, an intermediate `OrderItem` model is usually more appropriate because order-specific information such as quantity, unit price, discount, etc. needs to be stored.

Example:

```
```python  
class OrderItem(models.Model):
 order = models.ForeignKey(
 Order,
 on_delete=models.CASCADE
)
 product = models.ForeignKey(
 Product,
 on_delete=models.PROTECT
)
 quantity = models.PositiveIntegerField()
 unit_price = models.DecimalField(
...
```

```
max_digits=10,
decimal_places=2
)
...

Relationship Shortcut
```

```
```text
```

One One

OneToOneField

One Many

ForeignKey

Many Many

ManyToManyField

```
...  
  
---
```

```
# Part 4: Django Forward & Reverse Relationships
```

Example models:

```
```python
```

```
class Author(models.Model):
```

```
 name = models.CharField(max_length=100)
```

```
 email = models.EmailField(unique=True)
```

```
class Tag(models.Model):
```

```
name = models.CharField(max_length=50)
```

```
class Post(models.Model):
```

```
title = models.CharField(max_length=200)
```

```
content = models.TextField()
```

```
author = models.ForeignKey(
```

```
Author,
```

```
on_delete=models.CASCADE,
```

```
related_name='posts'
```

```
)
```

```
tags = models.ManyToManyField(
```

```
Tag,
```

```
related_name='posts'
```

```
)
```

```
created_at = models.DateTimeField(
```

```
auto_now_add=True
```

```
)
```

```
class Comment(models.Model):
```

```
post = models.ForeignKey(
```

```
Post,
```

```
on_delete=models.CASCADE,
```

```
related_name='comments'
```

```
)
```

```
content = models.TextField()
```

```
created_at = models.DateTimeField(
 auto_now_add=True
)
```

```
...
```

```

```

# Forward Relationship

**\*\*Forward\*\*** means following the field defined on the current model.

#### Post Author

```
```python
```

```
post.author
```

```
...
```

Comment Post

```
```python
```

```
comment.post
```

```
...
```

#### Post Tags

```
```python
```

```
post.tags.all()
```

```
...
```

```
---
```

Reverse Relationship

****Reverse**** means accessing objects from the other side of a relationship.

Author Posts

Because:

```
```python
author = models.ForeignKey(
 Author,
 related_name='posts'
)
```
```

We can use:

```
```python
author.posts.all()
```
```

Post Comments

```
```python
post.comments.all()
```
```

Tag Posts

```
```python
tag.posts.all()
```
```

Without `related\_name`

For a ForeignKey without `related_name`, Django normally provides a reverse manager such as:

```
```python
author.post_set.all()
```
```

With `related_name`:

```
```python
author.posts.all()
```
```

The second form is usually cleaner and easier to read.

Reverse Query Using Lookups

Suppose:

```
```text
Author

Post

Comment
```
```

Find comments on posts written by John:

```
```python
Comment.objects.filter(
 post__author__name='John'
```

)

...

Here:

```
```text
```

```
post__author__name
```

```
```
```

is a relationship lookup chain.

---

# Forward vs Reverse

| Relation | Query | Type |
|----------|-------|------|
|----------|-------|------|

|      |        |                            |         |
|------|--------|----------------------------|---------|
| Post | Author | <code>`post.author`</code> | Forward |
|------|--------|----------------------------|---------|

|        |      |                                   |         |
|--------|------|-----------------------------------|---------|
| Author | Post | <code>`author.posts.all()`</code> | Reverse |
|--------|------|-----------------------------------|---------|

|      |      |                                |         |
|------|------|--------------------------------|---------|
| Post | Tags | <code>`post.tags.all()`</code> | Forward |
|------|------|--------------------------------|---------|

|     |      |                                |         |
|-----|------|--------------------------------|---------|
| Tag | Post | <code>`tag.posts.all()`</code> | Reverse |
|-----|------|--------------------------------|---------|

|         |      |                             |         |
|---------|------|-----------------------------|---------|
| Comment | Post | <code>`comment.post`</code> | Forward |
|---------|------|-----------------------------|---------|

|      |         |                                    |         |
|------|---------|------------------------------------|---------|
| Post | Comment | <code>`post.comments.all()`</code> | Reverse |
|------|---------|------------------------------------|---------|

---

# Part 5: Basic Django ORM Queries

## Create

```
```python
```

```
author = Author.objects.create(
```

```
name='Faruk',
```

```
email='faruk@example.com'
```

```
)
```

```
...
```

```
```python
```

```
tag1 = Tag.objects.create(name='Python')
```

```
tag2 = Tag.objects.create(name='Django')
```

```
...
```

```
```python
```

```
post = Post.objects.create(
```

```
title='My First Post',
```

```
content='Content here',
```

```
author=author
```

```
)
```

```
...
```

Add Many-to-Many relationships:

```
```python
```

```
post.tags.add(tag1, tag2)
```

```
...
```

Create a comment:

```
```python
```

```
comment = Comment.objects.create(
```

```
post=post,
```

```
content='Nice post!'
```

```
)
```

```
...
```

```
---
```

```
# Read
```

```
```python
```

```
author_posts = author.posts.all()
```

```
...
```

```
```python
```

```
comments = post.comments.all()
```

```
...
```

```
```python
```

```
tag_posts = tag1.posts.all()
```

```
...
```

```
```python
```

```
post_author = post.author
```

```
...
```

```
---
```

```
# Filter
```

```
```python
```

```
Post.objects.filter(author=author)
```

```
...
```

```
```python
```

```
Post.objects.filter(tags=tag1)
```

```
...
```

```
```python
```

```
Comment.objects.filter(post=post)
```

```
```
```

```
---
```

```
# `__str__()`
```

For a readable representation in Django Admin and the shell:

```
```python
```

```
class Post(models.Model):
```

```
...
```

```
def __str__(self):
```

```
 return self.title
```

```
```
```

```
---
```

```
# Part 6: Django Query Optimization
```

```
## What is Query Optimization?
```

```
**Query Optimization** means improving database access by reducing unnecessary queries,  
unnecessary data loading, and expensive database operations.
```

```
### Main Goals
```

```
```text
```

```
Fewer unnecessary queries
```

```
+
```

```
Less unnecessary data
```

+

Efficient database operations

=

Better performance

...

---

# N+1 Query Problem

Consider:

```
```python
```

```
products = Product.objects.all()
```

```
for product in products:
```

```
    print(product.category.name)
```

```
```
```

Possible query pattern:

```
```text
```

```
1 query  Products
```

```
N queries Categories
```

```
```
```

Total:

```
```text
```

```
N + 1 queries
```

```
```
```

This is the **N+1 Query Problem**.

---

# Solution: `select\_related()`

For ForeignKey or OneToOne relationships:

```
```python
products = Product.objects.select_related(
    'category'
)
```
```

Then:

```
```python
for product in products:
    print(product.category.name)
```
```

`select\_related()` uses SQL JOINS to retrieve single-valued related objects efficiently.

#### Best For

```
```text
ForeignKey
OneToOneField
```
```

---

# `prefetch\_related()`

For Many-to-Many and reverse relationships:

```

```python
posts = Post.objects.prefetch_related(
    'tags'
)
```

```

Then:

```

```python
for post in posts:
    for tag in post.tags.all():
        print(tag.name)
```

```

Django generally uses separate queries and combines the results in Python.

#### Best For

```

```text

```

ManyToMany

Reverse ForeignKey

Other multi-valued relationships

```

```

```

```

```

# `select\_related()` vs `prefetch\_related()`

| Feature  | `select_related()` | `prefetch_related()` |
|----------|--------------------|----------------------|
| -----    | -----              | -----                |
| Main use | FK / OneToOne      | M2M / Reverse        |

| Technique | SQL JOIN | Separate queries + Python-side association |

| Typical query pattern | Often 1 query | Usually 2+ queries |

| Best for | Single-valued relations | Multi-valued relations |

---

# Combined Optimization

```
```python
```

```
posts = (  
    Post.objects  
        .select_related('author')  
        .prefetch_related('tags')  
)
```

```
```
```

Here:

```
```text
```

```
Author select_related()  
Tags prefetch_related()  
```
```

---

# Nested Relationship

```
```python
```

```
comments = Comment.objects.select_related(  
    'post__author'  
)
```

...

Then:

```
```python
for comment in comments:
 print(
 comment.content,
 comment.post.title,
 comment.post.author.name
)
```
```

This can retrieve comments, their posts, and their authors efficiently in a single SQL query.

Nested Prefetch

```
```python
authors = Author.objects.prefetch_related(
 'posts__tags'
)
```
```

Conceptually:

```
```text
```

Author

Posts

Tags

---

Django will typically use multiple queries to efficiently populate this relationship chain.

---

## # Part 7: Important Django Query Optimization Tools

### ## 1. `only()`

Load only selected fields initially:

```
```python
products = Product.objects.only(
    'id',
    'name',
    'price'
)
```
```

### ### Important

If a deferred field is accessed later, Django may issue another query.

---

### # 2. `defer()`

Avoid loading specific fields initially:

```
```python
products = Product.objects.defer(
    'description'
)
```

This can be useful for large fields.

3. `values()`

Returns dictionary-like results:

```
```python
products = Product.objects.values(
 'id',
 'name'
)
```
```

Example:

```
```python
[
 {'id': 1, 'name': 'Phone'},
 {'id': 2, 'name': 'Laptop'},
]
```
```

4. `values\_list()`

Returns selected values as tuples, or a flat list when `flat=True` is used:

```
```python
```

```
names = Product.objects.values_list(
 'name',
 flat=True
)
...
```

Result:

```
``python
['Phone', 'Laptop', 'Monitor']
...

```

# 5. `exists()`

Use `exists()` when you only need to know whether matching records exist:

```
``python
if Product.objects.filter(
 price=100
).exists():
 print('Found')
...

```

# 6. `count()`

Count database rows:

```
``python
count = Product.objects.count()
```

---

---

## # 7. `iterator()`

Can help control memory usage when iterating through a large `QuerySet`:

```
```python
for product in Product.objects.all().iterator():
    print(product.name)
```
```

### ### Important

`iterator()` is not automatically faster in every situation. Normal `QuerySet` caching may be preferable when repeated iteration is needed.

---

## # 8. `update()`

Perform a direct database update:

```
```python
Product.objects.filter(
    id=1
).update(price=200)
```
```

This does not call the model instance's `save()` method.

---

## # 9. `F()` Expression

Perform calculations at the database level:

```
```python
from django.db.models import F

Product.objects.update(
    stock=F('stock') - 1
)
```
```

This avoids reading the old value into Python before calculating the new value.

It is especially useful for concurrency-sensitive counters.

---

# 10. `Q()`

Build complex queries:

```
```python
from django.db.models import Q

products = Product.objects.filter(
    Q(price__lt=100) |
    Q(stock__gt=10)
)
```
```

This represents an OR condition.

---

## # 11. `annotate()`

Add calculated information to each result:

```
```python
from django.db.models import Count

products = Product.objects.annotate(
    total_variants=Count('variants')
)
```
```

Then:

```
```python
for product in products:
    print(
        product.name,
        product.total_variants
    )
```
```

---

## # 12. `aggregate()`

Calculate a summary for the whole QuerySet:

```
```python
from django.db.models import Sum

result = Product.objects.aggregate(
    total_price=Sum('price')
)
```

```
)  
...
```

Common aggregate functions:

```
```text  
Sum
Avg
Min
Max
Count
...
```

```

```

### # 13. Indexing

Indexes can improve queries that frequently filter, search, or order by specific fields.

Example:

```
```python  
class Product(models.Model):  
    name = models.CharField(  
        max_length=100,  
        db_index=True  
    )  
...
```

Important

Do not add indexes to every field blindly.

Indexes:

- * Can improve reads
- * Consume storage
- * Can increase insert/update/delete cost

Choose indexes based on actual query patterns.

14. ``select_for_update()``

Used to lock selected database rows during a transaction:

```
```python
from django.db import transaction

with transaction.atomic():
 product = (
 Product.objects
 .select_for_update()
 .get(id=1)
)
```
```

Useful for concurrency-sensitive operations.

15. ``in_bulk()``

Retrieve multiple objects by primary key as a dictionary:

```
```python
products = Product.objects.in_bulk(
[1, 2, 3]
)
```
```

16. `distinct()`

Remove duplicate results:

```
```python
products = Product.objects.distinct()
```
```

This can be useful with queries involving joins that produce duplicate rows.

17. `prefetch\_related\_objects()`

Prefetch relationships on already-loaded model instances:

```
```python
from django.db.models import prefetch_related_objects

prefetch_related_objects(
products,
'variants'
)
```
```

18. `explain()`

View the database execution plan:

```
```python
Product.objects.all().explain()
```
```

Useful for:

- * Query analysis
- * Index analysis
- * Performance debugging

19. Raw SQL

For advanced cases, Django supports raw SQL:

```
```python
Product.objects.raw(
'SELECT * FROM shop_product'
)
```
```

Important

Use raw SQL when necessary. Prefer Django ORM when it can express the query clearly and efficiently.

20. Pagination

Large datasets should not always be loaded at once.

```
```python
from django.core.paginator import Paginator

products = Product.objects.all()

paginator = Paginator(
 products,
 10
)

page_number = request.GET.get('page')

page = paginator.get_page(
 page_number
)
```
```

21. Caching

Frequently accessed expensive data can be cached.

```
```python
from django.core.cache import cache

data = cache.get('products')

if data is None:
 data = list(
```

```
Product.objects.values(
 'id',
 'name'
)
)
```

```
cache.set(
 'products',
 data,
 60
)
...
```

Caching can reduce repeated database hits.

---

# 22. `union()`

Combine compatible QuerySets:

```
```python  
qs1.union(qs2)  
...
```

23. `intersection()`

Return common results:

```
```python  
qs1.intersection(qs2)
```

...

---

# 24. `Subquery()`

Useful for advanced filtering and annotations.

```
```python
```

```
from django.db.models import OuterRef, Subquery
```

```
latest_price = (
```

```
Product.objects
```

```
.filter(category=OuterRef('pk'))
```

```
.order_by('-id')
```

```
.values('price')[:1]
```

```
)
```

...

Part 8: Django Model Design for Performance

Example:

```
```python
```

```
class Category(models.Model):
```

```
name = models.CharField(
```

```
max_length=100,
```

```
db_index=True
```

```
)
```

```
class Product(models.Model):
 category = models.ForeignKey(
 Category,
 on_delete=models.CASCADE,
 related_name='products'
)

 name = models.CharField(
 max_length=200,
 db_index=True
)

 price = models.DecimalField(
 max_digits=10,
 decimal_places=2
)

 created_at = models.DateTimeField(
 auto_now_add=True
)

class Variant(models.Model):
 product = models.ForeignKey(
 Product,
 on_delete=models.CASCADE,
 related_name='variants'
)

 size = models.CharField(max_length=50)
```

```
color = models.CharField(max_length=50)

stock = models.IntegerField(default=0)

...
```

### Important

Do not add `db\_index=True` to every field.

Choose indexes based on:

```
```text
```

Filtering

Searching

Ordering

Joining

Actual query patterns

```
```
```

```

```

# Part 9: Bad vs Optimized View

## Bad Example

```
```python
```

```
def product_list(request):
```

```
    products = Product.objects.all()
```

```
    data = []
```

```
    for product in products:
```

```
        variants = Variant.objects.filter(
```

```
            product=product
```

```

)

data.append({
  'name': product.name,
  'category': product.category.name,
  'variants': variants
})

return render(
  request,
  'shop.html',
  {'data': data}
)
...

```

Problems

Potentially:

```
``text
```

1 query Products

N queries Categories

N queries Variants

```
...
```

This creates unnecessary database work and can produce an N+1 pattern.

```
---
```

Optimized View

```

```python
def product_list(request):
 products = (
 Product.objects
 .select_related('category')
 .prefetch_related('variants')
)

 return render(
 request,
 'shop.html',
 {'products': products}
)
```

```

Here:

```

```text
Product Category
select_related()

Product Variants
prefetch_related()
```

```

Template

```

```django
{% for product in products %}

```

```
{{ product.name }}
```

```
{{ product.category.name }}
```

```
{% for variant in product.variants.all %}
```

```
{{ variant.size }} - {{ variant.color }}
```

```
{% endfor %}
```

```
{% endfor %}
```

```
...
```

Because the related data was prefetched/selected in advance, unnecessary N+1 queries can be avoided.

```

```

# Part 10: Important Note About `only()`

A query such as:

```
```python
```

```
products = (
```

```
Product.objects
```

```
.select_related('category')
```

```
.prefetch_related('variants')
```

```
.only(
```

```
'id',
```

```
'name',
```

```
'price',
```

```
'category__name'
```

```
)  
)  
...
```

can be valid, but `only()` should be used carefully with relationships.

If a field required later is deferred, Django may need additional queries.

For initial optimization, this is often easier to reason about:

```
```python  
products = (
 Product.objects
 .select_related('category')
 .prefetch_related('variants')
)
...
```

Then use `only()` after profiling shows that reducing selected columns is beneficial.

```

```

## # Part 11: Django Query Optimization Checklist

### ## 1. Avoid unnecessary queries inside loops

Bad:

```
```python  
for product in products:  
    Variant.objects.filter(  
        product=product  
    )
```

```
...
```

Better:

```
```python
products = Product.objects.prefetch_related(
 'variants'
)
```
```

```
---
```

2. ForeignKey / OneToOne

Use when appropriate:

```
```python
select_related()
```
```

```
---
```

3. ManyToMany / Reverse Relationship

Use when appropriate:

```
```python
prefetch_related()
```
```

```
---
```

4. Select only required data

Possible tools:

```
```text
```

```
values()
```

```
values_list()
```

```
only()
```

```
defer()
```

```
```
```

```
---
```

5. Existence check

```
```python
```

```
.exists()
```

```
```
```

```
---
```

6. Count rows

```
```python
```

```
.count()
```

```
```
```

```
---
```

7. Bulk operations

```
```text
```

```
bulk_create()
```

```
bulk_update()
```

```
update()
```

```
QuerySet.delete()
```

```
...
```

```

```

```
8. Database-side calculation
```

```
```python
```

```
F()
```

```
...
```

```
---
```

```
## 9. Complex conditions
```

```
```python
```

```
Q()
```

```
...
```

```

```

```
10. Aggregation
```

```
```text
```

```
annotate()
```

```
aggregate()
```

```
...
```

```
---
```

```
## 11. Indexing
```

Use appropriate indexes based on actual query patterns.

12. Large datasets

Consider:

```text

Pagination

iterator()

Batch processing

```

13. Frequently accessed expensive data

Consider:

```text

Caching

```

14. Query analysis

Use:

```text

Django Debug Toolbar

.explain()

Database query logs

```

Part 12: E-commerce Example

Suppose:

```text

Category

Product

Variant

```

Models:

```python

```
class Category(models.Model):
```

```
 name = models.CharField(max_length=100)
```

```
class Product(models.Model):
```

```
 category = models.ForeignKey(
```

```
 Category,
```

```
 on_delete=models.CASCADE,
```

```
 related_name='products'
```

```
)
```

```
 name = models.CharField(max_length=200)
```

```
 price = models.DecimalField(
```

```
 max_digits=10,
```

```
 decimal_places=2
```

)

```
class Variant(models.Model):
 product = models.ForeignKey(
 Product,
 on_delete=models.CASCADE,
 related_name='variants'
)
```

```
size = models.CharField(max_length=50)
color = models.CharField(max_length=50)
...
```

Optimized query:

```
```python
products = (
    Product.objects
    .select_related('category')
    .prefetch_related('variants')
)
...

```

Why?

```
```text
Product Category

select_related()

Product Variants
```

prefetch\_related()

...

---

# Part 13: Most Important Query Optimization Concepts

```text

N+1 Problem

select_related()

prefetch_related()

...

```text

Unnecessary fields

values()

values\_list()

only()

defer()

...

```text

Many inserts

bulk_create()

...

```text

Many updates

bulk\_update()

update()

...

```text

Database-side calculation

F()

...

```text

Complex conditions

Q()

...

```text

Per-row calculated data

annotate()

...

```text

Whole-query summary

aggregate()

...

```text

Frequently filtered/searched fields

Indexing

...

```text

Repeated expensive queries

Caching

```

```text

Concurrent updates

transaction.atomic()

select\_for\_update()

F()

```

```text

Query analysis

Debug Toolbar

.explain()

```

Part 14: Final Interview Revision

DBMS

```text

Redundancy

Same information stored unnecessarily multiple times.

Inconsistency

Same logical data has different values.

Anomaly

Problem during INSERT, UPDATE, or DELETE.

Normalization

Organizing data to reduce redundancy and anomalies.

Denormalization

Intentionally introducing some redundancy for specific performance needs.

Integrity

Data remains valid and follows defined constraints.

Consistency

Database remains in a valid logical state.

...

---

# Django Relationships

```text

One-to-One

OneToOneField

One-to-Many / Many-to-One

ForeignKey

Many-to-Many

ManyToManyField

...

Forward vs Reverse

```text

post.author

Forward

author.posts.all()

Reverse

comment.post

Forward

post.comments.all()

Reverse

```

Query Optimization

```text

ForeignKey / OneToOne

select\_related()

ManyToMany / Reverse FK

prefetch\_related()

Existence

exists()

Count

`count()`

Specific fields

`values()`

`values_list()`

`only()`

`defer()`

Bulk insert

`bulk_create()`

Bulk update

`bulk_update()`

Direct update

`update()`

Database calculation

`F()`

Complex filtering

`Q()`

Per-object calculation

`annotate()`

Whole-query calculation

`aggregate()`

Large dataset

pagination / iterator / batching

Repeated expensive data

cache

Query execution plan

explain()

Concurrency

transaction.atomic()

select\_for\_update()

...

---

# Golden Rules

#### Rule 1

Avoid unnecessary database queries inside loops.

#### Rule 2

Use ``select_related()`` for appropriate ForeignKey and OneToOne relationships.

#### Rule 3

Use ``prefetch_related()`` for appropriate ManyToMany and reverse/multi-valued relationships.

#### Rule 4

Use bulk operations when processing many records.

#### Rule 5

Use indexes based on real query patterns.

#### Rule 6

Use pagination or batching for large datasets.

#### ### Rule 7

Use caching for frequently requested expensive data.

#### ### Rule 8

Measure performance instead of guessing.

Useful tools:

``text

Django Debug Toolbar

Query logging

.explain()

...

#### ### Rule 9

Do not blindly use ``select_related()``, ``prefetch_related()``, ``only()``, or indexes.

Consider:

``text

Relationship type

+

Query pattern

+

Data size

+

Database workload

+

Actual performance

...

#### ### Rule 10

The goal of optimization is not simply to reduce query count.

A good optimization considers:

``text

Query count

+

Query cost

+

Database workload

+

Memory usage

+

Response time

+

Maintainability

...